

**МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)**

**Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»**

**Лабораторная работа №1
по курсу «Операционные системы»**

**Выполнил: В. С. Колобов
Группа: М8О-207Б-24
Преподаватель: Е. С. Миронов**

Москва, 2025

Условие

Цель работы: Приобретение практических навыков в управлении процессами в операционной системе и обеспечении обмена данными между процессами посредством каналов (pipe).

Задание: Составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними в одной из двух операционных систем. В результате работы программа (основной процесс) должен создать для решения задачи один или несколько дочерних процессов. Взаимодействие между процессами осуществляется через системные сигналы/события и/или каналы (pipe). Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы.

Вариант: 16

Описание варианта 16: Родительский процесс создает дочерний процесс. Первой строкой пользователь в консоль родительского процесса вводит имя файла, которое будет использовано для открытия File с таким именем на запись. Родительский процесс принимает от пользователя строки произвольной длины и пересылает их в pipe1. Процесс child проверяет строки на валидность правилу. Если строка соответствует правилу, то она выводится в стандартный поток вывода дочернего процесса, иначе в pipe2 выводится информация об ошибке. Родительский процесс полученные от child ошибки выводит в стандартный поток вывода. Правило проверки: строка должна оканчиваться на «.» или «;».

Метод решения

Программа реализует межпроцессное взаимодействие по схеме «родитель-потомок» с использованием двух неименованных каналов (pipe). Архитектура программы следует паттерну абстракции операционной системы, где системно-зависимые вызовы инкапсулированы в отдельный модуль.

Алгоритм работы:

1. Родительский процесс запрашивает у пользователя имя выходного файла
2. Создаются два канала: pipe1 (родитель → потомок) и pipe2 (потомок → родитель)
3. С помощью системного вызова `fork()` создается дочерний процесс
4. В дочернем процессе:
 - `stdin` перенаправляется на чтение из pipe1
 - `stdout` перенаправляется в выходной файл
 - `stderr` перенаправляется на запись в pipe2
 - Загружается и выполняется программа child
5. Родительский процесс читает строки от пользователя и отправляет их в pipe1
6. Дочерний процесс проверяет каждую строку:
 - Если строка оканчивается на '.' или ';' → выводит в `stdout` (записывается в файл)
 - Иначе → выводит сообщение об ошибке в `stderr` (отправляется в pipe2)

7. Родительский процесс читает сообщения об ошибках из `pipe2` и выводит их пользователю
8. После завершения ввода (`Ctrl+D`) родитель ожидает завершения дочернего процесса

Описание программы

Программа разделена на несколько модулей для обеспечения модульности и абстракции платформы.

Файловая структура:

- `parent.c` — основной процесс, управляет созданием потомка и каналов
- `child.c` — дочерний процесс, выполняет проверку строк
- `os.h` — абстрактный интерфейс системных вызовов
- `os_linux.c` — конкретная реализация для Linux

Основные типы данных:

```
typedef int pipe_t; // Тип для дескриптора канала
typedef int pid_t;  // Тип для идентификатора процесса
```

Функции интерфейса `os.h`:

- `CreatePipe()` — создание неименованного канала
- `CloneProcess()` — создание дочернего процесса
- `Exec()` — замена образа процесса
- `LinkFDtoIN()/LinkFDtoOUT()/LinkFDtoERR()` — перенаправление потоков
- `OpenObject()` — открытие файла
- `WritePipe()/ReadPipe()` — запись/чтение из канала
- `WaitProcess()` — ожидание завершения процесса
- `ClosePipe()` — закрытие дескриптора

Используемые системные вызовы Linux:

- `pipe()` — создание канала для межпроцессного взаимодействия
- `fork()` — создание идентичной копии процесса
- `dup2()` — дублирование файловых дескрипторов для перенаправления потоков
- `execv()` — замена текущего образа процесса новым
- `open()` — открытие/создание файла
- `read()/write()` — чтение/запись данных

- `close()` — закрытие файлового дескриптора
- `wait()` — ожидание изменения состояния процесса

Обработка ошибок: Программа проверяет возвращаемые значения всех системных вызовов и функций. При обнаружении ошибок выводится понятное сообщение с помощью `error()` и программа завершается с ненулевым кодом возврата.

Результаты

Была разработана программа, реализующая межпроцессное взаимодействие по схеме «родитель-потомок» с использованием двух неименованных каналов (`pipe`) в операционной системе Linux.

Ключевые особенности решения:

1. Программа использует интерфейс `'os.h'`, который инкапсулирует системные вызовы, что теоретически позволяет легко портировать код на другие платформы.
2. Реализовано два канала: - `'pipe1'`: передача строк от родителя к потомку - `'pipe2'`: передача сообщений об ошибках от потомка к родителю
3. Все файловые дескрипторы и каналы закрываются своевременно, предотвращая утечки ресурсов и зависания программы.
4. Программа проверяет возвращаемые значения всех системных вызовов и выводит понятные сообщения об ошибках.
5. Родительский и дочерний процессы представлены разными исполняемыми файлами, что соответствует требованиям задания.

Выводы

В ходе выполнения лабораторной работы были успешно приобретены практические навыки управления процессами в операционной системе Linux и организации межпроцессного взаимодействия через неименованные каналы (`pipe`).

Исходная программа

Заголовочный файл os.h

```
1 | #pragma once
2 |
3 | #ifdef _WIN32
4 | #else
5 | #include <errno.h>
6 | #include <fcntl.h>
7 | #include <stdio.h>
8 | #include <stdlib.h>
9 | #include <string.h>
10 | #include <sys/wait.h>
11 | #include <unistd.h>
12 |
13 | typedef int pipe_t;
14 | typedef int pid_t;
15 |
16 | int CreatePipe(pipe_t new_pipe[2]);
17 | pid_t CloneProcess();
18 | int Exec(const char* path, char* argv[]);
19 | int LinkFDtoIN(int fd);
20 | int LinkFDtoOUT(int fd);
21 | int LinkFDtoERR(int fd);
22 | int OpenObject(const char* path, int flags, int mod);
23 | int WaitProcess();
24 | int ClosePipe(pipe_t pipe);
25 | int WritePipe(pipe_t pipe, void* buffer, int bytes);
26 | int ReadPipe(pipe_t pipe, void* buffer, int bytes);
27 | #endif
```

Листинг 1: *Сюда описание*

Реализация для Linux

```
1 | #include "os.h"
2 | #include <stdio.h>
3 | #include <stdlib.h>
4 | #include <string.h>
5 | #include <unistd.h>
6 | #include <sys/wait.h>
7 | #include <fcntl.h>
8 |
9 | int CreatePipe(pipe_t new_pipe[2]) { return pipe(new_pipe); }
10 |
11 | pid_t CloneProcess() { return fork(); }
12 |
13 | int Exec(const char* path, char* argv[]) {
14 |     return execv(path, argv);
15 | }
16 |
17 | int LinkFDtoIN(int fd) {
18 |     return dup2(fd, STDIN_FILENO);
19 | }
```

```

20
21 int LinkFDtoOUT(int fd) {
22     return dup2(fd, STDOUT_FILENO);
23 }
24
25 int LinkFDtoERR(int fd) {
26     return dup2(fd, STDERR_FILENO);
27 }
28
29 int OpenObject(const char* path, int flags, int mod) {
30     return open(path, flags, mod);
31 }
32
33 int WaitProcess() { return wait(NULL); }
34
35 int ClosePipe(pipe_t pipe) { return close(pipe); }
36
37 int WritePipe(pipe_t pipe, void* buffer, int bytes) {
38     return write(pipe, buffer, bytes);
39 }
40
41 int ReadPipe(pipe_t pipe, void* buffer, int bytes) {
42     return read(pipe, buffer, bytes);
43 }

```

Листинг 2: *Сюда описание*

Родительский процесс

```

1  #include "os.h"
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <string.h>
5
6  #define BUFFER_SIZE 512
7
8  int main() {
9      int pipe_to_child[2];
10     int pipe_from_child[2];
11     pid_t child_pid;
12     char filename[BUFFER_SIZE];
13     int file_fd;
14
15     printf("Enter output filename: ");
16     if(!fgets(filename, BUFFER_SIZE, stdin)) {
17         perror("Input error");
18         exit(1);
19     }
20     filename[strcspn(filename, "\n")] = 0;
21
22     file_fd = OpenObject(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);
23     if(file_fd == -1) {
24         perror("File opening failed");
25         exit(1);
26     }
27

```

```

28     if(CreatePipe(pipe_to_child) == -1 || CreatePipe(pipe_from_child) == -1) {
29         perror("Pipe creation failed");
30         exit(1);
31     }
32
33     child_pid = CloneProcess();
34     if(child_pid == -1) {
35         perror("Process creation failed");
36         exit(1);
37     }
38
39     if(child_pid == 0) {
40         ClosePipe(pipe_to_child[1]);
41         ClosePipe(pipe_from_child[0]);
42
43         LinkFDtoIN(pipe_to_child[0]);
44         ClosePipe(pipe_to_child[0]);
45
46         LinkFDtoOUT(file_fd);
47         ClosePipe(file_fd);
48
49         LinkFDtoERR(pipe_from_child[1]);
50         ClosePipe(pipe_from_child[1]);
51
52         Exec("./child", (char*[]){ "child", NULL });
53         perror("Execution failed");
54         exit(1);
55     } else {
56         ClosePipe(pipe_to_child[0]);
57         ClosePipe(pipe_from_child[1]);
58         ClosePipe(file_fd);
59
60         char buffer[BUFFER_SIZE];
61
62         while(fgets(buffer, BUFFER_SIZE, stdin)) {
63             WritePipe(pipe_to_child[1], buffer, strlen(buffer));
64         }
65         ClosePipe(pipe_to_child[1]);
66
67         int bytes_read;
68         while((bytes_read = ReadPipe(pipe_from_child[0], buffer, BUFFER_SIZE-1)) > 0) {
69             buffer[bytes_read] = '\0';
70             printf("Error from child: %s", buffer);
71         }
72         ClosePipe(pipe_from_child[0]);
73
74         WaitProcess();
75     }
76     return 0;
77 }

```

Листинг 3: *Сюда описание*

Дочерний процесс

```

1 | #include <stdio.h>
2 | #include <string.h>
3 | #include <stdlib.h>
4 |
5 | #define LINE_BUFFER 512
6 |
7 | int main() {
8 |     char line[LINE_BUFFER];
9 |
10 |    while(fgets(line, LINE_BUFFER, stdin)) {
11 |        int len = strlen(line);
12 |        if(len > 0 && line[len-1] == '\n') {
13 |            line[len-1] = '\0';
14 |            len--;
15 |        }
16 |
17 |        if(len > 0 && (line[len-1] == '.' || line[len-1] == ';')) {
18 |            printf("%s\n", line);
19 |        } else {
20 |            fprintf(stderr, "Invalid line: '%s' (must end with '.' or ';')\n", line);
21 |        }
22 |    }
23 |
24 |    return 0;
25 | }

```

Листинг 4: *Сюда описание*

Пример вывода программы

```

$ gcc -o parent parent.c os_linux.c
$ gcc -o child child.c
$ ./parent
Enter output filename: output.txt
Hello world.
This is valid;
This line is not valid
Another valid line.
^D
Error from child: Invalid line: 'This line is not valid'(must end with '.' or
';')

$ cat output.txt
Hello world.
This is valid;
Another valid line.

```

Системные вызовы (strace)

Для анализа системных вызовов можно использовать утилиту strace:

```
$ strace -f -o trace.log ./parent
```


В файле trace.log будут видны все системные вызовы, включая:

- `openat()` — открытие файла
- `pipe()` — создание каналов
- `fork()` — создание дочернего процесса
- `dup2()` — перенаправление дескрипторов
- `execve()` — запуск дочерней программы
- `read()/write()` — чтение/запись в каналы
- `wait4()` — ожидание завершения потомка

Вывод Strace:

```
$ strace ./parent
execve("./parent",["./parent"],0x7ffff13f8940 /* 66 vars */) = 0
write(1,"Enter output filename: ",23) = 23
read(0,"file_output\n",4096)      = 12
openat(AT_FDCWD,"file_output",O_WRONLY|O_CREAT|O_TRUNC,0644) = 3
pipe([4,5])                       = 0
pipe([6,7])                       = 0
clone(child_stack=NULL,flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,
child_tidptr=0x7f8a1d7d4a10) = 2487

[pid 2486] write(4,"helo world\n",11) = 11
[pid 2487] read(0,"helo world\n",512) = 11
[pid 2487] write(2,"Invalid line: 'helo world'(must end with '.'or ';'')\n",55)
= 55
[pid 2486] read(7,"Invalid line: 'helo world'(must end with '.'or ';'')\n",512)
= 55
[pid 2486] write(1,"Error from child: Invalid line: 'helo world'(must end
with '.'or ';'')\n",72) = 72

[pid 2486] write(4,"this is valid\n",14) = 14
[pid 2487] read(0,"this is valid\n",512) = 14
[pid 2487] write(2,"Invalid line: 'this is valid'(must end with '.'or ';'')\n",57)
= 57
[pid 2486] read(7,"Invalid line: 'this is valid'(must end with '.'or ';'')\n",512)
= 57
[pid 2486] write(1,"Error from child: Invalid line: 'this is valid'(must end
with '.'or ';'')\n",74) = 74
```